Interface Requirements for Automated Docketing and Filing in a Modernized CM/ECF

A practitioner's perspective for the case-management modernization program

Jackson Sanger — Independent Developer, ECFiler Project jackson@sang3r.com · github.com/jackson-jpeg/ecfiler · July 2026

Written to be forwardable inside the AO: no product pitch, every requirement anchored to a security or accuracy benefit, sources cited.

---

1. Who this is from, and why

I write in a personal capacity, from two vantage points. By profession I am a litigation docketing specialist: PACER and CM/ECF are my daily working environment, at the frequency and failure-mode exposure that only docketing work produces. No employer is named or represented in this paper, and nothing here reports on how any particular firm operates. Separately, as an independent open-source developer, I build ECFiler, a tool that prepares federal filings and helps a filer check them against local requirements before submission. It works against the Judiciary's documented public interfaces — the PACER Authentication API and the Case Locator API — and against CM/ECF itself through the filer's own browser session, on the filer's own machine, under the filer's own credentials, which the tool never transmits or stores on a server.

I am not an attorney and not a filing agent, and ECFiler has never filed on behalf of a client. What I can describe precisely is, first, the mechanics of high-volume docketing work as practiced daily, and second, what the sanctioned interfaces do and do not offer to anyone trying to build compliant automation on top of them — and where those gaps push integrators toward practices the Judiciary has itself warned against.

This paper describes what professional filing operations need from the interfaces of a modernized case-management system. It is written now because the requirements for the replacement system are being set now: the Judicial Conference outlined an accelerated modernization in March 2026 and approved funding for it in June 2026, with early components already being piloted at six courts.[1][2]

The core argument: **most of what makes today's third-party ecosystem risky is not misbehavior — it is the absence of interfaces designed for delegation.** Every gap in the sanctioned surface pushes legal professionals toward workarounds the Judiciary itself has warned against. Closing those gaps is a security measure first and a convenience second.

2. The status quo pushes credentials in the wrong direction

The filing login is the attorney's signature under local rules. Yet in practice, attorneys do not operate CM/ECF alone: docketing departments, paralegals, and filing services do much of the work. Because the system has no concept of a delegated, limited-purpose credential, the working reality in firms of every size is shared attorney passwords — the exact practice the Administrative Office's July 10, 2023 memorandum and the court notices that followed it warned filers about, in the context of third-party service providers with access to sealed and restricted material.[3]

The 2023 guidance was right about the risk and the industry adjusted — major vendors stopped storing ECF credentials. But the underlying demand did not go away, because the work still has to be done by people other than the credential holder. A modernized system can resolve this correctly, at the platform layer, instead of leaving it to policy memoranda:

> **Requirement 1 — Delegated application credentials, distinct from the attorney's signature credential.** Scoped, revocable, per-application tokens (issued under an attorney's or firm's account) that can prepare and stage work, with the attorney-held signature credential required for the act of submission itself. OAuth-style delegation is the established pattern. The security benefit is direct: it replaces password sharing — invisible, unscoped, unrevocable — with delegation that is visible, scoped, and auditable by the courts.
>
> **Requirement 2 — Scoped tokens that cannot reach sealed or restricted material.** Delegation must be narrower than the delegator's own access. A token used by software should be issuable with an explicit ceiling: no sealed documents, no restricted-case content, regardless of what the underlying account may access. The August 2025 incident made concrete what is at stake in sealed-material exposure;[4] scoped tokens turn "trust every integration with everything" into "trust each integration with exactly what it needs."

3. Filing-interface requirements

These are the properties that determine whether filing automation is safe. Each one removes a class of error that today lands on clerks' offices as correction work, or on attorneys as missed deadlines and duplicate fee charges.

> **Requirement 3 — Idempotent submission.** A filing request must carry a client-generated identifier such that a retry after a timeout cannot docket twice. Today, a connection failure at the submit step leaves the filer guessing whether the entry exists — the cause of duplicate entries clerks must strike and duplicate fees that take weeks to reverse.
>
> **Requirement 4 — The event catalog as data.** Each court's event codes, categories, per-event document requirements, and fee consequences, published as a versioned, machine-readable feed (the court-lookup JSON/XML feeds are the right precedent[5]). Event selection is the highest-judgment step in docketing; mis-selection is the most common filing defect. A published catalog

> lets software present accurate choices to the human making that judgment — it does not replace the judgment.
>
> **Requirement 5 — Deterministic fee quoting.** Given an event and case posture, the system should quote the fee before submission, in data. Fee surprises are a correction-queue burden for courts and a reconciliation burden for firms.
>
> **Requirement 6 — Structured filing results and NEFs.** The confirmation returned at submission — and the NEF stream generally — as structured data (case, entry number, document identifiers, timestamps), not only formatted text. Every docketing operation in the country parses NEF emails with regular expressions today; every parser is a silent failure waiting to misfile a deadline.
>
> **Requirement 7 — Sandbox parity.** A test environment that mirrors production interfaces and per-court configuration, available to any registered developer (the QA PACER environment is the right precedent). Integrations tested against production-shaped systems fail in production less; courts absorb fewer of those failures.
>
> **Requirement 8 — Attestation at the point of submission.** The interface should make the human act explicit: submission requires the signature credential (Req. 1) and the system records what was displayed to the attesting filer. This gives courts a durable answer to "who signed this?" that is stronger, not weaker, than today's shared-password reality.

4. Why interfaces beat prohibitions

The Judiciary has twice responded to third-party integration pressure with warnings — credential sharing (2023) and scripted retrieval load (the long-standing request that bulk automated queries run 6 p.m.–6 a.m. CT). Both warnings are reasonable, and both are stopgaps: they manage symptoms of missing interfaces. Integrations that identify themselves, authenticate with scoped tokens, and use supported endpoints can be rate- limited, audited, and — when they misbehave — individually revoked. Anonymous scripted traffic through user-facing pages can only be detected and blocked. A modernized system that offers the sanctioned path will find that most integrators prefer it; those that remain outside it become legitimately distinguishable.

The bankruptcy experience is the proof inside the Judiciary's own record: Case Upload and the NextGen XML case-opening interfaces gave petition-preparation software a sanctioned channel, and structured bankruptcy filing became routine without credential-sharing controversy.[6] District civil and criminal practice has no equivalent; that asymmetry, not any difference in demand, is why the gray-zone tooling concentrates there.

5. What I am asking for

1. That filing-interface requirements for the modernized system be gathered from docketing professionals as a distinct user class — the people who operate these systems at the highest frequency and see their failure modes first.
2. That delegated, scoped application credentials (Req. 1–2) be treated as a security deliverable of modernization, on equal footing with encryption and access control — because they are the structural fix for credential sharing.
3. A pointer to the right channel for this input, if it is not the EPA Public User Group or the developer mailbox.

I am glad to elaborate on any of this, with specifics from daily practice, at any level of technical depth that is useful.

---

1. uscourts.gov, “Judges Outline Accelerated Modernization of Case Management System,” Mar. 10, 2026. https://www.uscourts.gov/data-news/judiciary-news/2026/03/10/judges-outline-accelerated-modernization-case-management-system ↩

2. uscourts.gov, “Judiciary Approves Funding for Case Management and Public Access Modernization,” June 26, 2026. https://www.uscourts.gov/data-news/judiciary-news/2026/06/26/judiciary-approves-funding-case-management-and-public-access-modernization ↩

3. See, e.g., D.P.R. Notice from the Clerk 23-06 (citing the AO’s July 10, 2023 memorandum); D.C. Cir., “CM/ECF — Access to Case Information and Documents by Third-Party Services” (2023). ↩

4. uscourts.gov, “Cybersecurity Measures Strengthened in Light of Attacks on Judiciary’s Case Management System,” Aug. 7, 2025 (describing “escalated cyberattacks of a sophisticated and persistent nature” and new procedures to restrict access to sealed documents). https://www.uscourts.gov/data-news/judiciary-news/2025/08/07/cybersecurity-measures-strengthened-light-attacks-judiciarys-case-management-system ↩

5. pacer.uscourts.gov court CM/ECF lookup feeds (data.json / data.xml). ↩

6. PACER Developer Resources: Bankruptcy Case Upload; NextGen CM/ECF XML Case Opening and Docketing documentation. https://pacer.uscourts.gov/file-case/developer-resources ↩